

Práctica 2. Trabajando con los datos

26/01/2025

Objetivos

Los objetivos que se persiguen en esta práctica son:

1. Aprender a almacenar y recuperar datos en R
2. Ser capaz de importar y exportar datos de otros sistemas
3. Organizar y manipular datos
4. Entender la necesidad de limpiar los datos

Almacenamiento y recuperación de datos en R

R dispone de un formato nativo `RData` en el que almacenar objetos. La función que se usa para guardar objetos es `save` y para cargarlos `load`. Otra opción es almacenar y recuperar el entorno de trabajo completo, para lo que se emplean las funciones `save.image` y `load.image`. Esto se realiza en RStudio a través de la pestaña **Environment**.

Ejemplos `save`

```
# Crear objetos de ejemplo
vector <- c(10, 20, 30, 40, 50)
matriz <- matrix(1:6, nrow = 2)
dataframe <- data.frame(
  name = c("Ana", "Juan", "Luis"),
  age = c(22, 33, 44)
)

# Guardar el data frame en un archivo .RData
save(dataframe, file = "alumnos.RData")

# Guardar varios objetos en un archivo .RData
save(vector, matriz, dataframe, file = "mis_datos.RData")
```

Ejemplo `load`

```
# Cargar los objetos desde el archivo .RData
load("mis_datos.RData")

# Verificar los objetos cargados
print(vector)
```

```
## [1] 10 20 30 40 50
```

```
print(matriz)
```

```
##      [,1] [,2] [,3]
## [1,]    1    3    5
## [2,]    2    4    6
```

```
print(dataframe)
```

```
##   name age
## 1  Ana  22
## 2  Juan  33
## 3  Luis  44
```

Ejemplo save.image

```
# Crear algunos objetos de ejemplo
a <- 5
b <- c(1, 2, 3)
c <- data.frame(x = 1:3, y = c("A", "B", "C"))

# Guardar todo el entorno de trabajo
save.image(file = "mi_entorno_completo.RData")
```

Ejemplo load.image

```
# Cargar todo el entorno de trabajo desde el archivo .RData
load("mi_entorno_completo.RData")

# Verificar los objetos cargados
print(a)
```

```
## [1] 5
```

```
print(b)
```

```
## [1] 1 2 3
```

```
print(c)
```

```
##   x y
## 1 1 A
## 2 2 B
## 3 3 C
```

Importación y exportación de otras fuentes de datos

No es habitual que los datos se creen en R, mas bien al contrario, son creados por otros sistemas y almacenados en alguno de los formatos estándar de almacenamiento de datos. Los más conocidos y extendidos son los ficheros de texto, los ficheros CSV y los ficheros EXCEL.

En **R** se pueden importar datos desde múltiples fuentes, incluyendo consulta a bases de datos y peticiones AJAX. En esta práctica, importaremos y exportaremos datos de ficheros de texto, ficheros CSV y ficheros EXCEL.

Ficheros Texto

Puedes usar la función `read.table()` para importar ficheros de texto y la función `write.table()` para generarlos.

Ejemplos

```
# Importar un fichero de texto
#datos <- read.table("ruta/al/archivo.txt", header = TRUE, sep = "\t")
```

ruta/al/archivo.txt: Especifica la ruta del fichero de texto.

header: Indica si el fichero tiene encabezados en la primera fila (TRUE o FALSE).

sep: Especifica el delimitador de los datos (en este caso, tabulaciones).

```
# Exportar datos a un fichero de texto  
#write.table(datos, file = "ruta/al/archivo_exportado.txt",  
# sep = "\t", row.names = FALSE, col.names = TRUE)
```

datos: Es el nombre del dataframe que quieres exportar.

file: Especifica la ruta y el nombre del fichero de texto que se creará.

sep: Especifica el delimitador que usarás en el fichero de texto.

row.names: Indica si quieres incluir los nombres de las filas (TRUE o FALSE).

col.names: Indica si quieres incluir los nombres de las columnas (TRUE o FALSE).

Ficheros CSV

Puedes usar la función `read.csv()` para importar ficheros CSV y la función `write.csv()` para generarlos.

```
#Importar ficheros CSV  
  
#datos <- read.csv("ruta/al/archivo.csv", header = TRUE, sep = ",")
```

`ruta/al/archivo.csv:` Especifica la ruta del fichero CSV.

header: Indica si el fichero tiene encabezados en la primera fila (TRUE o FALSE).

sep: Especifica el delimitador de los datos (en este caso, una coma).

```
#Exportar ficheros CSV  
#write.csv(datos, file = "ruta/al/archivo_exportado.csv", row.names = FALSE)
```

datos: Es el nombre del dataframe que quieres exportar.

file: Especifica la ruta y el nombre del fichero CSV que se creará.

row.names: Indica si quieres incluir los nombres de las filas (TRUE o FALSE).

Ejemplo Completo:

```
#Aquí tienes un ejemplo completo que muestra cómo importar y exportar ficheros CSV:  
  
# Importar un fichero CSV  
#datos_importados <- read.csv("ruta/al/archivo.csv", header = TRUE, sep = ",")  
  
# Realizar alguna operación con los datos (opcional)  
# Por ejemplo, seleccionar solo las columnas específicas  
#datos_seleccionados <- datos_importados[, c("Columna1", "Columna2")]  
  
# Exportar los datos seleccionados a un nuevo fichero CSV  
#write.csv(datos_seleccionados, file = "ruta/al/archivo_seleccionado.csv",  
# row.names = FALSE)
```

Ficheros EXCEL

Las funciones que hacen posible la importación y exportación de ficheros EXCEL no se encuentran en la base de **R**, siendo necesaria la utilización de paquetes adicionales, concretamente `readxl` y `writexl`. Previamente, recuerda que debes instalar estos paquetes y cargarlos antes de usarlos.

Ejemplo importación de datos

```
#Instalar el paquete readxl si no se ha hecho antes
install.packages("readxl", repos="https://cran.rediris.es/")
```

```
## package 'readxl' successfully unpacked and MD5 sums checked
## Warning: cannot remove prior installation of package 'readxl'
## Warning in file.copy(savedcopy, lib, recursive = TRUE): problema al copiar
## D:\Mis programas\R-4.4.1\library\00LOCK\readxl\libs\x64\readxl.dll a D:\Mis
## programas\R-4.4.1\library\readxl\libs\x64\readxl.dll: Permission denied
## Warning: restored 'readxl'
##
## The downloaded binary packages are in
## C:\Users\UJA\AppData\Local\Temp\RtmpotHGsu\downloaded_packages
```

```
#Cargar el paquete readxl:
library(readxl)
```

```
## Warning: package 'readxl' was built under R version 4.4.2
```

```
#Importar un fichero Excel:
#datos <- read_excel("ruta/al/archivo.xlsx", sheet = 1)
```

ruta/al/archivo.xlsx: Especifica la ruta completa del fichero Excel que deseas importar.

sheet: Indica la hoja del fichero Excel que deseas importar. Puedes especificar el número de la hoja (por ejemplo, sheet = 1) o su nombre (por ejemplo, sheet = "Hoja1").

Adicionalmente, puedes utilizar otros argumentos útiles como:

range: Especifica un rango de celdas para leer (por ejemplo, range = "A1:D10").

col_types: Define los tipos de columnas a leer (por ejemplo, col_types = c("text", "numeric", "date", "text")).

na: Especifica los valores que deben ser tratados como NA (por ejemplo, na = "NA").

Ejemplo exportación de datos

```
#Instalar el paquete writexl si no se ha hecho antes:
```

```
install.packages("writexl", repos="https://cran.rediris.es/")
```

```
## package 'writexl' successfully unpacked and MD5 sums checked
##
## The downloaded binary packages are in
## C:\Users\UJA\AppData\Local\Temp\RtmpotHGsu\downloaded_packages
```

```
#Cargar el paquete writexl:
library(writexl)
```

```
## Warning: package 'writexl' was built under R version 4.4.2
```

```
# Exportar datos a un fichero Excel
#write_xlsx(datos, path = "ruta/al/archivo_exportado.xlsx")
```

datos: Es el nombre del dataframe que deseas exportar.

path: Especifica la ruta completa y el nombre del fichero Excel que se creará.

Organización y manipulación de los datos

Una vez que disponemos de un conjunto de datos, lo primero que debemos hacer es familiarizarnos con ellos: ver su estructura, las variables, los valores que toman, si hay datos faltantes, etc.

Conociendo la estructura de los datos

Ejemplo

```
#cargamos los datos
```

```
library(readxl)
practica2 <- read_excel("practica2.xlsx")
```

```
#Mostramos las diez primeras lineas
head(practica2)
```

```
## # A tibble: 6 x 30
##   CodMun Municipio `Extensión superficial (Km2). 2019` `Perímetro (m). 2019`
##   <dbl> <chr>                <dbl>                <dbl>
## 1  4001 Abla                    45.2                    46584.
## 2  4002 Abrucena                83.6                    58589.
## 3  4003 Adra                    89.6                    51598.
## 4  4004 Albanchez               34.9                    26873.
## 5  4005 Alboloduy               69.7                    47845.
## 6  4006 Albox                   168.                    61462.
## # i 26 more variables: `Altitud sobre el nivel del mar (m). 2019` <dbl>,
## # `Número de núcleos que componen el municipio. 2023` <dbl>,
## # `Distancia a la capital (Km). 2019` <dbl>, `Población total. 2023` <dbl>,
## # `Población. Hombres. 2023` <dbl>, `Población en núcleos. 2023` <dbl>,
## # `Población en diseminados. 2023` <dbl>, `Edad media. 2022` <dbl>,
## # `Porcentaje de población menor de 20 años. 2023` <dbl>,
## # `Porcentaje de población mayor de 65 años. 2023` <dbl>, ...
```

```
#Mostramos las dimensiones de los datos
```

```
dim(practica2)#785 municipios y para cada uno de ellos se dispone de 96 variables
```

```
## [1] 785 30
```

```
#Mostramos los nombres de las variables
```

```
names(practica2)
```

```
## [1] "CodMun"
## [2] "Municipio"
## [3] "Extensión superficial (Km2). 2019"
## [4] "Perímetro (m). 2019"
## [5] "Altitud sobre el nivel del mar (m). 2019"
## [6] "Número de núcleos que componen el municipio. 2023"
## [7] "Distancia a la capital (Km). 2019"
## [8] "Población total. 2023"
## [9] "Población. Hombres. 2023"
## [10] "Población en núcleos. 2023"
## [11] "Población en diseminados. 2023"
## [12] "Edad media. 2022"
## [13] "Porcentaje de población menor de 20 años. 2023"
## [14] "Porcentaje de población mayor de 65 años. 2023"
## [15] "Variación relativa de la población en diez años (%). 2012-2022"
```

```
## [16] "Número de extranjeros. 2022"
## [17] "Principal procedencia de los extranjeros residentes. 2022"
## [18] "Emigraciones. 2022"
## [19] "Inmigraciones. 2022"
## [20] "Nacimientos. 2023"
## [21] "Defunciones. 2023"
## [22] "Matrimonios. 2023"
## [23] "Alcaldía. 2023"
## [24] "Hoteles. 2023"
## [25] "Hostales y pensiones. 2023"
## [26] "Plazas en hoteles. 2023"
## [27] "Plazas en hostales y pensiones. 2023"
## [28] "Tasa municipal de desempleo (%). 2023"
## [29] "Renta bruta media. 2022"
## [30] "Renta disponible media. 2022"
```

```
#Examinamos la estructura de las variables que componen el conjunto de datos
str(practica2)
```

```
## tibble [785 x 30] (S3: tbl_df/tbl/data.frame)
##   $ CodMun                                : num [1:785] 4001 4002 4003 4004 4005 ...
##   $ Municipio                             : chr [1:785] "Abla" "Abrucena" "Adra" ...
##   $ Extensión superficial (Km2). 2019      : num [1:785] 45.2 83.6 89.6 34.9 69.1 ...
##   $ Perímetro (m). 2019                   : num [1:785] 46584 58589 51598 26812 26812 ...
##   $ Altitud sobre el nivel del mar (m). 2019 : num [1:785] 843 972 4 469 383 ...
##   $ Número de núcleos que componen el municipio. 2023 : num [1:785] 3 4 20 5 2 7 3 6 1 1 ...
##   $ Distancia a la capital (Km). 2019      : num [1:785] 61.2 63.2 52.7 73.2 30.1 ...
##   $ Población total. 2023                  : num [1:785] 1259 1274 25178 680 680 ...
##   $ Población. Hombres. 2023               : num [1:785] 652 674 12875 367 295 ...
##   $ Población en núcleos. 2023             : num [1:785] 1211 1242 25050 625 58 ...
##   $ Población en diseminados. 2023         : num [1:785] 44 33 145 57 12 ...
##   $ Edad media. 2022                      : num [1:785] 49.5 49.9 40.7 55.1 50.1 ...
##   $ Porcentaje de población menor de 20 años. 2023 : num [1:785] 13.7 13 21.5 10.1 13.1 ...
##   $ Porcentaje de población mayor de 65 años. 2023 : num [1:785] 28.9 28.4 15.4 40.1 30.1 ...
##   $ Variación relativa de la población en diez años (%). 2012-2022: num [1:785] -14.9 -10.2 2.7 -5 -8 ...
##   $ Número de extranjeros. 2022           : num [1:785] 117 116 3657 277 34 ...
##   $ Principal procedencia de los extranjeros residentes. 2022 : chr [1:785] "Marruecos" "Marruecos" "Marruecos" ...
##   $ Emigraciones. 2022                    : num [1:785] 77 90 1005 70 28 ...
##   $ Inmigraciones. 2022                   : num [1:785] 89 143 972 51 21 963 ...
##   $ Nacimientos. 2023                     : num [1:785] 8 7 203 2 1 108 8 0 1 ...
##   $ Defunciones. 2023                     : num [1:785] 25 14 214 14 12 123 9 ...
##   $ Matrimonios. 2023                     : num [1:785] 8 4 68 3 1 39 3 0 0 3 ...
##   $ Alcaldía. 2023                        : chr [1:785] "PARTIDO SOCIALISTA OBRERO ...
##   $ Hoteles. 2023                         : chr [1:785] "-" "-" "*" "-" ...
##   $ Hostales y pensiones. 2023             : chr [1:785] "*" "-" "*" "-" ...
##   $ Plazas en hoteles. 2023                : chr [1:785] "-" "-" "*" "-" ...
##   $ Plazas en hostales y pensiones. 2023   : chr [1:785] "*" "-" "*" "-" ...
##   $ Tasa municipal de desempleo (%). 2023 : chr [1:785] "19.3" "17.8" "16.7" ...
##   $ Renta bruta media. 2022               : chr [1:785] "18289" "17047" "18930 ...
##   $ Renta disponible media. 2022          : chr [1:785] "15843" "15133" "16300 ...
```

Modificación del nombre de las variables

En **R** se puede modificar el nombre de una o varias variables, para ello haremos uso de la función `names`.

Ejemplo

```
#De uno en uno
names(practica2)[5] <- "Altitud19"

#Varios de una vez
names(practica2)[c(3, 4, 6, 7, 8)] <-
  c("Extension19", "Perimetro19", "Nucleos23", "DistanciaCapital19", "Poblacion23")
```

Generación de nuevas variables

La generación de una nueva variable en un conjunto de datos consiste en añadir una variable al mismo y rellenarla con información. Generalmente, esta información se obtendrá mediante una expresión matemática en la que estarán involucradas una o más variables. No obstante, también se puede generar una variable cualitativa a partir de otras variables.

Ejemplos

```
#Creamos la variable TasaHombres
practica2$TasaHombres <- practica2$`Población. Hombres. 2023` / practica2$Poblacion23

#Otra forma de hacer lo mismo es
practica2 <- within(practica2, TasaHombres <- `Población. Hombres. 2023` / Poblacion23)

#Generamos una variable con valor condicional
practica2$Provincia =
  ifelse(4000 <= practica2$CodMun & practica2$CodMun <= 4999, "Almeria",
  ifelse(11000 <= practica2$CodMun & practica2$CodMun <= 11999, "Cádiz",
  ifelse(14000 <= practica2$CodMun & practica2$CodMun <= 14999, "Córdoba",
  ifelse(18000 <= practica2$CodMun & practica2$CodMun <= 18999, "Granada",
  ifelse(21000 <= practica2$CodMun & practica2$CodMun <= 21999, "Huelva",
  ifelse(23000 <= practica2$CodMun & practica2$CodMun <= 23999, "Jaén",
  ifelse(29000 <= practica2$CodMun & practica2$CodMun <= 29999, "Málaga",
  ifelse(41000 <= practica2$CodMun & practica2$CodMun <= 41999, "Sevilla", NA))))))
```

Recodificación

La recodificación consiste en generar una variable cualitativa a partir de otra preexistente cambiando los valores de la misma. El procedimiento sería similar a la creación.

Ejemplos

```
# Crear un vector de ejemplo
valores <- c(1, 2, 3, 4, 5)

# Crear un vector de recodificación inicial
recodificado <- as.character(valores)

# Recodificar valores directamente
recodificado[valores <= 2] <- "Bajo"
recodificado[valores == 3] <- "Medio"
recodificado[valores >= 4] <- "Alto"
```

En el paquete `car` está disponible la función `recode` que simplifica el proceso de recodificación.

```
install.packages("car", repos="https://cran.rediris.es/")
```

```
## package 'car' successfully unpacked and MD5 sums checked
##
```

```
## The downloaded binary packages are in
## C:\Users\UJA\AppData\Local\Temp\RtmpotHGsu\downloaded_packages
library(car)
```

```
## Warning: package 'car' was built under R version 4.4.2
## Cargando paquete requerido: carData
## Warning: package 'carData' was built under R version 4.4.2
```

```
practica2$Provincia<-recode(practica2$CodMun,
  "4000:4999='Almería' ;
  11000:11999='Cádiz' ;
  14000:14999='Córdoba' ;
  18000:18999='Granada';
  21000:21999='Huelva' ;
  23000:23999='Jaén' ;
  29000:29999='Málaga' ;
  41000:41999='Sevilla'" , as.factor=TRUE)
```

Selección de un conjunto de variables

Cuando se dispone de un conjunto de datos con varias variables es posible generar un nuevo conjunto de datos solo con las variables que se deseen.

```
#Crear un nuevo conjunto de datos que solo contenga las variables CodMun,
# Extension19, Perimetro19 y Altitud23

datos <- practica2[, c("CodMun", "Extension19", "Perimetro19", "Altitud19", "Poblacion23")]

#Otra forma de conseguir lo mismo es mediante la posición de la columna
datos <- practica2[, c(1, 3, 4, 5, 8)]
```

Filtrado

Filtrar un conjunto de datos consiste en seleccionar los individuos o elementos que cumplen una determinada condición. Así pues, de un filtrado obtendremos un subconjunto de los datos en el que todos los elementos cumplen dicha condición.

Ejemplos

```
#Dados los datos practica2 se desea conocer el número de municipios de la provincia de Almería
datosAlmeria <- practica2[practica2$Provincia == "Almería", ]
nrow(datosAlmeria)

## [1] 103

#El filtrado puede ser más completo usando condicionales
#Obtenemos los municipios de Almería con más de 10000 habitantes

datosAlmeriaMayor10000<-practica2[practica2$Provincia == "Almería" & practica2$Poblacion23 > 10000, ]
nrow(datosAlmeriaMayor10000)

## [1] 14

head(datosAlmeriaMayor10000)

## # A tibble: 6 x 32
##   CodMun Municipio Extension19 Perimetro19 Altitud19 Nucleos23
```



```
##      <dbl> <chr>                <dbl>      <dbl>      <dbl>      <dbl>
## 1    4003 Adra                    89.6      51598.      4         20
## 2    4006 Albox                   168.      61462.     424        7
## 3    4013 Almería                 296.     127327.     20         17
## 4    4029 Berja                   186.      66421.     335        12
## 5    4035 Cuevas del Almanzora    264.      75791.      95         26
## 6    4902 El Ejido                225.      73856.      72         14
## # i 26 more variables: DistanciaCapital19 <dbl>, Poblacion23 <dbl>,
## #   `Población. Hombres. 2023` <dbl>, `Población en núcleos. 2023` <dbl>,
## #   `Población en diseminados. 2023` <dbl>, `Edad media. 2022` <dbl>,
## #   `Porcentaje de población menor de 20 años. 2023` <dbl>,
## #   `Porcentaje de población mayor de 65 años. 2023` <dbl>,
## #   `Variación relativa de la población en diez años (%). 2012-2022` <dbl>,
## #   `Número de extranjeros. 2022` <dbl>, ...
```

Ordenación de datos

En ciertos procedimientos estadísticos se precisa que los datos se encuentren ordenados en función de una determinada variable. Para ello se emplea la función `order`.

```
#Ordenación creciente
datos_ordenados <- practica2[order(practica2$Poblacion23), ]
head(datos_ordenados)
```

```
## # A tibble: 6 x 32
##   CodMun Municipio      Extension19 Perimetro19 Altitud19 Nucleos23
##   <dbl> <chr>          <dbl>      <dbl>      <dbl>      <dbl>
## 1    4026 Benitagla      6.39      14147.      945         1
## 2   21027 Cumbres de Enmedio 13.6      16370.      589         1
## 3    4033 Castro de Filabres 29.2      29595.      951         1
## 4    4009 Alcudia de Monteagud 15.5      22476.     1012         1
## 5   18121 Lobras         16.0      22139.      918         2
## 6    4023 Beires        38.8      37727      930         1
## # i 26 more variables: DistanciaCapital19 <dbl>, Poblacion23 <dbl>,
## #   `Población. Hombres. 2023` <dbl>, `Población en núcleos. 2023` <dbl>,
## #   `Población en diseminados. 2023` <dbl>, `Edad media. 2022` <dbl>,
## #   `Porcentaje de población menor de 20 años. 2023` <dbl>,
## #   `Porcentaje de población mayor de 65 años. 2023` <dbl>,
## #   `Variación relativa de la población en diez años (%). 2012-2022` <dbl>,
## #   `Número de extranjeros. 2022` <dbl>, ...
```

```
#Ordenación decreciente
datos_ordenados <- practica2[order(practica2$Poblacion23, decreasing = TRUE), ]
head(datos_ordenados)
```

```
## # A tibble: 6 x 32
##   CodMun Municipio      Extension19 Perimetro19 Altitud19 Nucleos23
##   <dbl> <chr>          <dbl>      <dbl>      <dbl>      <dbl>
## 1   41091 Sevilla       141.      74216.      5         1
## 2   29067 Málaga       395.     145429.      6         14
## 3   14021 Córdoba     1255.     232920.     111        69
## 4   18087 Granada       88.1      58993.     680         5
## 5   11020 Jerez de la Frontera 1189.     273180.      45         27
## 6    4013 Almería       296.     127327.     20         17
## # i 26 more variables: DistanciaCapital19 <dbl>, Poblacion23 <dbl>,
## #   `Población. Hombres. 2023` <dbl>, `Población en núcleos. 2023` <dbl>,
```

```
## # `Población en diseminados. 2023` <dbl>, `Edad media. 2022` <dbl>,
## # `Porcentaje de población menor de 20 años. 2023` <dbl>,
## # `Porcentaje de población mayor de 65 años. 2023` <dbl>,
## # `Variación relativa de la población en diez años (%). 2012-2022` <dbl>,
## # `Número de extranjeros. 2022` <dbl>, ...
```

Limpiar los datos

En ocasiones, los datos facilitados presentan problemas estructurales que debemos corregir antes de empezar a explotarlos. Uno de los más comunes es la existencia de datos faltantes. En este curso de iniciación, optaremos por eliminar las filas o columnas con datos faltantes, o bien emplearemos una técnica de imputación de datos. La imputación consiste en reemplazar los valores faltantes por valores estimados. Por ejemplo, se puede utilizar la media, la mediana o la moda de la variable. Otras técnicas más avanzadas son la imputación por los k vecinos más cercanos. Nosotros utilizaremos la media.

Otras veces, el tipo de los datos no coincide con su contenido y habrá que cambiar al tipo adecuado. Esto puede deberse a que haya algún dato faltante o a que se hayan codificado los datos faltantes con alguna secuencia de caracteres.

#Si observamos la variable `Renta bruta media. 2022` observamos que los valores están entre comillas y hay municipios que tienen como valor `-'`.

#Vamos a convertir en numéricos las cifras que se encuentran entre comillas.

```
practica2$`Renta bruta media. 2022` <- as.numeric(practica2$`Renta bruta media. 2022`)
```

```
## Warning: NAs introducidos por coerción
```

```
head(practica2$`Renta bruta media. 2022`)
```

```
## [1] 18289 17047 18938    NA    NA 22117
```

Eliminamos filas con datos faltantes usando na.omit()

```
practica2_sin_na <- na.omit(practica2)
```

Mostramos el data frame sin filas con datos faltantes

```
head(practica2_sin_na)
```

```
## # A tibble: 6 x 32
```

```
##   CodMun Municipio      Extension19 Perimetro19 Altitud19 Nucleos23
```

```
##   <dbl> <chr>          <dbl>      <dbl>      <dbl>      <dbl>
```

```
## 1  4001 Abia          45.2      46584.      843         3
```

```
## 2  4002 Abrucena      83.6      58589.      972         4
```

```
## 3  4003 Adra          89.6      51598.         4        20
```

```
## 4  4006 Albox        168.      61462.      424         7
```

```
## 5  4011 Alhama de Almería 26.2      25588.      518         2
```

```
## 6  4013 Almería       296.     127327.      20        17
```

```
## # i 26 more variables: DistanciaCapital19 <dbl>, Poblacion23 <dbl>,
```

```
## # `Población. Hombres. 2023` <dbl>, `Población en núcleos. 2023` <dbl>,
```

```
## # `Población en diseminados. 2023` <dbl>, `Edad media. 2022` <dbl>,
```

```
## # `Porcentaje de población menor de 20 años. 2023` <dbl>,
```

```
## # `Porcentaje de población mayor de 65 años. 2023` <dbl>,
```

```
## # `Variación relativa de la población en diez años (%). 2012-2022` <dbl>,
```

```
## # `Número de extranjeros. 2022` <dbl>, ...
```

#Imputación de datos faltantes

```
practica2$`Renta bruta media. 2022`[is.na(practica2$`Renta bruta media. 2022`)] =
    mean(practica2$`Renta bruta media. 2022`, na.rm = TRUE)

# Mostramos la hoja de datos con los valores imputados
head(practica2$`Renta bruta media. 2022`)
```

```
## [1] 18289.00 17047.00 18938.00 20243.84 20243.84 22117.00
```

EJERCICIOS

1. A partir del conjunto de datos `practica2`, genere un nuevo conjunto de datos llamado `practica2_1` que solo contenga las variables `codMun` y `Municipio`. Exporte el conjunto de datos a un fichero llamado `practica2_1.csv`.
2. Cambie el nombre de todas las variables del conjunto de datos `practica 2` por uno más corto. Exporte los datos a un fichero de texto llamado `practica2_2.txt` utilizando como separador `;`.
3. Genere una nueva variable `TasaMujeres23` a partir de `Poblacion23` y `Población. Hombres. 2023`. Almacene los datos resultantes con las últimas modificaciones en un fichero EXCEL llamado `practica2_3.xlsx`.
4. Cree un nuevo conjunto de datos que contenga los municipios de Jaén y las variables `Municipio`, `Extension19`, `Perimetro19`, `Altitud19`, `DistanciaCapital19`, `Poblacion23`, `EdadMedia22`). Almacénelo en un fichero EXCEL llamado `practica2_4.xlsx`. Cuando finalice, borre todos los objetos del espacio de trabajo.
5. Cargue el fichero `xlsx` creado en el ejercicio anterior y muestre en pantalla la `EdadMedia22` de los municipios que están a una altura superior a 850 m.
6. A partir del conjunto de datos `practica2` visualice los valores que toma la variable `Principal procedencia de los extranjeros residentes. 2022`. ¿Qué observa en los datos? Cambie los valores faltantes por `NA`.
7. Visualice la variable `Renta disponible media. 2022`. ¿El tipo de datos es coherente? En caso negativo, modifíquelo. ¿Hay datos faltantes? En caso afirmativo, impute dichos valores utilizando la mediana (función `median`).
8. Una vez imputados los datos faltantes, genere una variable factor llamada `renta` con los siguientes valores:

Renta disponible por habitante	Renta
0 - 14999.99	Menos de 15000
15000 - 19999.99	Entre 15000 y menos 20000
20000 - 24999.99	Entre 20000 y menos 25000
25000 - 29999.99	Entre 25000 y menos 30000
30000 - 9999999999	30000 o superior

9. Guarde los datos resultantes en un fichero `.Rdata`.